c:\hpc-23-feb-2026\.\14\eg117.cpp

```
#include<iostream>
using namespace std;
void sam(int &x)
{
cout<<x<<endl;
x=50;
}
int main()
{
sam(10);
return 0;
}
```

Thinking Machines Classroom Codes

c:\hpc-23-feb-2026\.\14\eg118.cpp

```
#include<iostream>
using namespace std;
void sam(int &&x) // && to capture rvalue reference
{
cout<<x<<endl;
}
int main()
{
sam(10);
return 0;
}
```

Thinking Machines Classroom Codes

c:\hpc-23-feb-2026\.\14\eg119.cpp

```
#include<iostream>
using namespace std;
class aaa
{
};
void do_something(aaa &&k)
{
}
int main()
{
aaa j;
do_something(j); // will not compile, j is of lvalue type
return 0;
}
```

c:\hpc-23-feb-2026\.\14\eg120.cpp

```
#include<iostream>
using namespace std;
class aaa
{
};
void do_something(aaa &&k)
{
}
int main()
{
aaa j;
do_something(move(j)); // move produces rvalue type
do_something(aaa()); // tmp object means rvalue type
return 0;
}
```

Thinking Machines Classroom Codes

c:\hpc-23-feb-2026\.\14\eg121.cpp

```
#include<iostream>
using namespace std;
class aaa
{
public:
aaa(){}
aaa(const aaa &)=delete; // copy constructor marked as deleted
aaa(aaa &&)
{
}
};
void do_something(aaa k)
{
}
int main()
{
aaa j;
do_something(j);
return 0;
}
```

c:\hpc-23-feb-2026\.\14\eg122.cpp

```
#include<iostream>
using namespace std;
class aaa
{
public:
aaa(){
cout<<"default"<<endl;
}
aaa(const aaa &)=delete; // copy constructor marked as deleted
aaa(aaa &&)
{
cout<<"moved"<<endl;
}
};
void do_something(aaa k)
{
}
int main()
{
aaa j;
do_something(move(j));
return 0;
}
```

c:\hpc-23-feb-2026\.\14\eg123.cpp

```
#include<iostream>
using namespace std;
class aaa
{
public:
aaa(){
cout<<"default"<<endl;
}
aaa(const aaa &)=delete; // copy constructor marked as deleted
aaa(aaa &&)
{
cout<<"moved"<<endl;
}
};
void do_something(aaa &&k)
{
}
int main()
{
aaa j;
do_something(move(j));
return 0;
}
```

c:\hpc-23-feb-2026\.\14\eg124.cpp

```
#include<future>
using namespace std;
int main()
{
promise<int> prm1;
promise<int> prm2;
prm2=prm1; // will not compile
return 0;
}
```

c:\hpc-23-feb-2026\.\14\eg125.cpp

```
#include<iostream>
#include<thread>
#include<future>
#include<chrono>
using namespace std;
int power(int num,int p)
{
if(p==0) return 1;
this_thread::sleep_for(chrono::seconds(1));
return (p==1)?num:num*power(num,p-1);
}
void promise_implementor(promise<int> &&prm,int n,int p)
{
try
{
int result=power(n,p);
prm.set_value(result);
}catch(...)
{
prm.set_exception(current_exception());
}
}
int main()
{
promise<int> prm;
future<int> fut=prm.get_future();
cout<<"great"<<endl;
promise_implementor(move(prm),5,3);
cout<<"cool"<<endl;
int x=fut.get();
cout<<x<<endl;
return 0;
}
```

c:\hpc-23-feb-2026\.\14\eg126.cpp

```
#include<iostream>
#include<thread>
#include<future>
#include<chrono>
using namespace std;
int power(int num,int p)
{
if(p==0) return 1;
this_thread::sleep_for(chrono::seconds(1));
return (p==1)?num:num*power(num,p-1);
}
void promise_implementor(promise<int> &&prm,int n,int p)
{
try
{
int result=power(n,p);
prm.set_value(result);
}catch(...)
{
prm.set_exception(current_exception());
}
}
int main()
{
promise<int> prm;
future<int> fut=prm.get_future();
cout<<"great"<<endl;
thread t(promise_implementor,move(prm),5,3);
cout<<"cool"<<endl;
int x=fut.get(); // may block using wait()
t.join(); // essential to release resources (rule, either call join or detach)
cout<<x<<endl;
return 0;
}
```

c:\hpc-23-feb-2026\.\14\eg127.cpp

```
#include<iostream>
#include<future>
#include<chrono>
#include<thread>
using namespace std;
int power(int n,int p)
{
if(p==0) return 1;
this_thread::sleep_for(chrono::seconds(1));
return (p==1)?n:n*power(n,p-1);
}
int main()
{
packaged_task task(power);
future<int> f=task.get_future();
cout<<"God is great 1234"<<endl;
thread t(move(task),5,3);
cout<<"God is great 5332"<<endl;
int r=f.get();
cout<<r<<endl;
t.join();
return 0;
}
```

Thinking Machines Classroom Codes

c:\hpc-23-feb-2026\.\14\eg128.cpp

```
#include<iostream>
#include<future>
#include<chrono>
#include<thread>
using namespace std;
int power(int n,int p)
{
if(p==0) return 1;
this_thread::sleep_for(chrono::seconds(1));
return (p==1)?n:n*power(n,p-1);
}
int main()
{
future<int> f=async(power,5,3);
// because of the above line, a separate thread may or may not get created,
// it depends on the implementation of async written for the platform
// default policy behind async may be async or deferred
// in next example, we will specify the policy to be async
cout<<"God is greaT"<<endl;
int x=f.get();
cout<<x<<endl;
return 0;
}
```

Thinking Machines Classroom Codes

c:\hpc-23-feb-2026\.\14\eg129.cpp

```
#include<iostream>
#include<future>
#include<chrono>
#include<thread>
using namespace std;
int power(int n,int p)
{
if(p==0) return 1;
this_thread::sleep_for(chrono::seconds(1));
return (p==1)?n:n*power(n,p-1);
}
int main()
{
future<int> f=async(launch::async,power,5,3); // create thread
cout<<"God is greaT"<<endl;
int x=f.get();
cout<<x<<endl;
return 0;
}
```

c:\hpc-23-feb-2026\.\14\eg130.c

```
#include<pthread.h>
#include<stdio.h>
void * do_something(void *ptr)
{
int j;
for(j=503;j<=604;++j) printf("%d ",j);
pthread_exit(nullptr);
printf("This will never print\n");
return nullptr; // this is useless
}
int main()
{
int u;
pthread_t tid; // for thread id
int i;
i=pthread_create(&tid,nullptr,do_something,nullptr);
if(i)
{
printf("Unable to create thread\n");
return 1; // instead of 0, I am puttin myself into the habit of returning 1
  // to say that all didn't go well
}
for(u=1;u<=50;++u) printf("%d ",u);
pthread_join(tid,nullptr);
printf("main is about to end\n");
return 0;
}
```

c:\hpc-23-feb-2026\.\14\eg131.c

```
#include<pthread.h>
#include<stdio.h>
typedef struct _targs
{
int start;
int end;
}targs;
void * do_something(void *ptr)
{
targs *m=(targs *)ptr;
int j;
for(j=m->start;j<=m->end;++j) printf("%d ",j);
pthread_exit(nullptr);
printf("This will never print\n");
return nullptr; // this is useless
}
int main()
{
targs a;
int u;
pthread_t tid; // for thread id
int i;
a.start=555;
a.end=666;
i=pthread_create(&tid,nullptr,do_something,&a);
if(i)
{
printf("Unable to create thread\n");
return 1; // instead of 0, I am puttin myself into the habit of returning 1
  // to say that all didn't go well
}
for(u=1;u<=50;++u) printf("%d ",u);
pthread_join(tid,nullptr);
printf("main is about to end\n");
return 0;
}
```

c:\hpc-23-feb-2026\.\13\eg112.cpp

```
#include<iostream>
#include<future>
#include<chrono>
#include<thread>
using namespace std;
int power(int n,int p)
{
if(p==0) return 1;
this_thread::sleep_for(chrono::seconds(1));
return (p==1)?n:n*power(n,p-1);
}
int main()
{
packaged_task task(power);
future<int> f=task.get_future();
cout<<"God is great 1234"<<endl;
task(5,3); // task is a callable object, separate thread comes in action over here
        // no separate thread will come, in action, explained in session 14
cout<<"God is great 5332"<<endl;
int r=f.get(); // join will get called for the separate thread which is behind the task
  // join will not get called, explained in session 14
cout<<r<<endl;
return 0;
}
```